

Mecanismo de Reprodução de Algoritmos Genéticos do MATLAB

Aluno: Leonardo Ronald Perin Rauta

Professor: Carlos Alberto Bavastri

1. Introdução

O Algoritmo Genético do MATLAB (ga) é um solver heurístico desenvolvido para problemas de otimização onde a função objetivo pode ser descontínua, estocástica ou altamente não-linear. Diferente de algoritmos clássicos vistos em aula, baseados em derivadas que geram um único ponto a cada iteração, o GA trabalha com uma população de pontos simultaneamente.

Os algoritmos genéticos (GA) são métodos inspirados na teoria da evolução de Charles Darwin. Os GA trabalham com uma População de possíveis soluções que evoluem ao longo do tempo, ou gerações. As melhores soluções sobrevivem e se reproduzem, gerando novas soluções (filhos) através de *crossover* e mutação. Isso é repetido até se obter uma geração ou valores que estão adequados à solução do problema proposto.

O GA é amplamente utilizado em otimização pois possui uma grande probabilidade de encontrar o mínimo global da função. Isso o difere dos algoritmos clássicos, vistos em aula, que precisam de funções modais, e, muitas vezes, acabam encontrando apenas o mínimo local da função.

Os GA utilizam, a partir de uma população inicial, três regras para criar a próxima geração populacional.

- Na regra da seleção são escolhidas as soluções (pais) que mais se adequam à necessidade do problema (*fitness*). Esses permanecem para a próxima geração.
- Na regra da mutação são aplicadas mudanças aleatórias para gerar uma mutação e garantir uma variabilidade genética, garantindo assim, uma diversidade.
- E na regra de *crossover*, são combinadas as melhores soluções para gerar uma nova geração mais adaptada à necessidade do problema. Quando se trabalham com GA em outros softwares ou então desenvolve algoritmos de GA, fora do MATLAB, uma das técnicas utilizadas é mesclar o primeiro *nibble*¹ de uma solução

1 Nibble – Combinação de 4 bits, ou, meio byte.

com o segundo *nibble* de outra solução, gerando assim, um novo “filho” para a nova geração, diferente da técnica utilizada pelo MATLAB.

2. GA no MATLAB

O algoritmo genético implementado pela função `ga` do MATLAB traz uma abordagem focada na resolução de problemas de engenharia. Em vez de utilizar a tradicional representação biológica binária (em que os indivíduos são formados por cadeias de zeros e uns), a ferramenta trabalha nativamente com a população estruturada em vetores de números reais. Devido a essa forma de trabalho vetorial, a regra de *crossover* no MATLAB não realiza a junção dos *nibbles*; ele utiliza funções algébricas e lógicas para combinar os pais.

Alguns dos métodos utilizados, que serão explicados posteriormente, são o *Crossover* Disperso, em que selecionam aleatoriamente de qual pai cada valor numérico inteiro será herdado; o *Crossover* Aritmético em que um filho é criado calculando literalmente a média ponderada das coordenadas dos pais.

Além disso, o MATLAB difere de grande parte da literatura clássica sobre GA ao adotar a estratégia de “*Crossover* ou Mutação”. A mutação, no MATLAB, não é aplicada em cima do filho recém-cruzado, ele adota nativamente a estratégia para que uma parte garantida da população passe intacta para a próxima geração. O restante da nova população é dividido estritamente entre filhos gerados apenas por *crossover* e filhos gerados apenas por mutação.

a. Fluxo de GA no MATLAB

De acordo com o manual, o algoritmo genético do MATLAB segue um roteiro estrito a cada iteração (geração) para otimizar o problema:

1. Criação da População Inicial: O algoritmo gera aleatoriamente a primeira população de indivíduos (pontos no espaço de busca), respeitando os limites (*bounds*) definidos pelo usuário.
2. Avaliação (*Scoring*): A função objetivo é calculada para cada indivíduo. O valor retornado é o score (nota) daquela solução.
3. Escalonamento (*Scaling*): O MATLAB não usa os scores puros para a seleção. Ele os converte em valores de expectativa (escalonamento). Isso impede que um indivíduo excepcionalmente bom domine a população logo

nas primeiras gerações, o que causaria convergência prematura para um mínimo local.

4. Seleção: Com base no escalonamento, o algoritmo seleciona membros da população atual para serem os Pais da próxima geração.
5. Reprodução: O MATLAB cria a nova geração através de três regras estritas:
 - Elite: Uma quantidade fixa das melhores soluções passa para a próxima geração sem sofrer nenhuma alteração.
 - Crossover: Filhos são criados combinando vetores de dois pais.
 - Mutação: Filhos são criados aplicando variações aleatórias a um único pai.
6. Substituição: A população atual é inteiramente substituída pela nova geração, e o ciclo se repete.

3. Funções nativas de *crossover* (**CrossoverFcn**)

Conforme já apresentado, o MATLAB opera com o tipo de população *doubleVector* (vetores de números reais), não realizando a manipulação direta de bits. A ferramenta fornece diversas funções e técnicas para realizar a combinação desses vetores de ponto flutuante:

a. **Crossover Disperso** (**crossoverscattered**)

Método do MATLAB para problemas que não possuem restrições lineares. Como o algoritmo opera com vetores de números reais, o método disperso não quebra os números ao meio; em vez disso, ele sorteia quais posições (coordenadas) do vetor serão herdadas de cada pai.

O algoritmo gera internamente um vetor binário aleatório (composto apenas por zeros e uns) do mesmo tamanho que o indivíduo. Esse vetor atua como uma máscara, onde a máscara apresentar o valor 1, o filho herda o valor exato daquela posição do Pai 1, caso contrário, herda do Pai 2.

Exemplo:

Pai 1: [1.5, 3.2, 8.4, 5.0]

Pai 2: [0.0, 9.9, 1.1, 2.2]

Máscara aleatória gerada: [1, 0, 0, 1]

Filho resultante: [1.5, 9.9, 1.1, 5.0]

b. Crossover Aritmético (*crossoverarithmetic*)

Diferente do método disperso, que apenas escolhe coordenadas inteiras de um ou de outro pai, o *crossover* aritmético cria um filho que é uma mistura matemática das posições dos pais. Ele calcula uma média ponderada entre os dois vetores pais.

O MATLAB gera um multiplicador aleatório (peso) e aplica uma combinação linear. Isso garante que o filho sempre nasça em um ponto intermediário no espaço de busca, estritamente entre a posição do Pai 1 e do Pai 2.

Exemplo:

Considere um peso gerado aleatoriamente de 0.5 (ou seja, o filho nascerá exatamente no ponto médio).

Pai 1: [2.0, 4.0]

Pai 2: [10.0, 10.0]

Cálculo: Filho = 0.5 x Pai_1 + (1 - 0.5) x Pai_2

Filho resultante: [6.0, 7.0]

c. Crossover Intermediário (*crossoverintermediate*)

Método padrão adotado pelo MATLAB quando o problema de otimização possui restrições lineares (inequações ou equações). Ele é uma variação mais expansiva do Crossover Aritmético.

Enquanto o Aritmético prende o filho estritamente no segmento de reta entre os dois pais, o Intermediário permite que o filho seja gerado além da posição dos pais, utilizando um parâmetro de proporção (*ratio*).

A fórmula básica é: Filho = Pai_1 + rand x Ratio x (Pai_2 - Pai_1).

Isso ajuda o algoritmo a não encolher a população excessivamente a cada geração.

Exemplo:

Assumindo um valor aleatório gerado de 0.8 e um *ratio* configurado para permitir extrapolação.

Pai 1: [1.0, 1.0]

Pai 2: [2.0, 2.0]

Cálculo: $\text{Filho} = \text{Pai_1} + \text{rand} \times \text{Ratio} \times (\text{Pai_2} - \text{Pai_1})$

Filho resultante: Pode nascer em [2.2, 2.2], indo além das coordenadas do Pai 2, expandindo a área de busca.

d. **Crossover Heurístico (crossoverheuristic)**

Este é um método de cruzamento que utiliza a avaliação da função objetivo para guiar a criação do filho. Em vez de operar de forma puramente cega e aleatória, ele verifica qual dos dois pais possui a melhor nota (*fitness*).

O algoritmo então traça um vetor direcional que sai do Pior Pai, passa pelo Melhor Pai e extrapola um pouco mais à frente. A premissa lógica é heurística: se mover da posição do Pior Pai para o Melhor Pai resultou em uma melhora na função, dar mais um passo nessa mesma direção tem grande chance de gerar um filho ainda mais apto.

Exemplo Prático:

Assumindo que o Pai 1 tem um valor de *fitness* melhor que o Pai 2, e um fator de passo (ratio) de 1.2.

Melhor Pai (Pai 1): [1.0, 1.0]

Pior Pai (Pai 2): [0.0, 0.0]

Cálculo: $\text{Filho} = \text{Pai_1} + 1.2 \times (\text{Pai_1} - \text{Pai_2})$

Filho resultante: [1.2, 1.2].

e. **Crossover de Ponto Único (crossoveringlepoint)**

Este método é a adaptação direta da literatura clássica de algoritmos genéticos (que cortavam o DNA binário) para o mundo dos vetores de números reais.

O MATLAB sorteia aleatoriamente um índice inteiro n dentro do tamanho do vetor (cromossomo). O filho é então construído copiando as coordenadas da primeira posição até o índice n do Pai 1, e as coordenadas restantes (do índice $n+1$ até o final) do Pai 2.

Exemplo:

Considere vetores de 5 variáveis e um ponto de corte sorteado no índice $n = 3$.

Pai 1: [1, 2, 3, 4, 5]

Pai 2: [9, 8, 7, 6, 5]

Ponto de Corte: Após o terceiro número.

Filho resultante: [1, 2, 3, 6, 5] (Herdou as 3 primeiras casas do Pai 1 e as 2 últimas do Pai 2).

4. Funções nativas de mutação (`MutationFcn`)

Assim como ocorre no cruzamento, a mutação no MATLAB não segue a abordagem biológica clássica de inverter um bit (trocar 0 por 1 e vice-versa). Como a população é composta por vetores de números reais, a mutação atua como uma perturbação matemática. O objetivo da mutação é injetar diversidade na população, tirando o algoritmo de mínimos locais e explorando novas áreas do espaço de busca.

a. Mutação Gaussiana (`mutationgaussian`)

É a função padrão do MATLAB para problemas sem restrições (*unconstrained*). Em vez de trocar o valor do indivíduo por um número completamente aleatório, o algoritmo adiciona um ruído estatístico a ele.

Para cada coordenada do vetor do pai escolhido para mutação, ele soma um valor aleatório retirado de uma distribuição normal (Gaussiana) centrada em zero. Um detalhe dessa função é que a variância desse ruído diminui conforme as gerações avançam. Nas primeiras gerações, os passos de mutação são largos para explorar o mapa; nas últimas gerações, os passos são pequenos, atuando apenas como um ajuste fino da solução.

Exemplo:

Considere um indivíduo que sofrerá mutação. O algoritmo sorteia ruídos pequenos baseados na distribuição Gaussiana atual.

Pai (antes da mutação): [2.50, 4.10, 10.00]

Ruído Gaussiano gerado: [0.15, -0.30, 0.05]

Cálculo: Filho = Pai + Ruído

Filho Mutante resultante: [2.65, 3.80, 10.05]

b. Mutação Uniforme (`mutationuniform`)

Este método é uma abordagem mais agressiva de exploração. Ele opera de forma semelhante ao sorteio de variáveis aleatórias simples. O algoritmo seleciona aleatoriamente uma (ou mais) posições do vetor do pai e substitui o valor original por um

número completamente novo, sorteado de forma uniforme dentro dos limites máximo e mínimo permitidos para aquela variável.

Diferente da Gaussiana, que apenas empurra o indivíduo um pouco para o lado, a mutação uniforme pode transportar uma coordenada do indivíduo para o outro extremo do espaço de busca.

Exemplo:

Assuma um problema onde as variáveis têm limite inferior de 0.0 e superior de 20.0. O algoritmo decide mutar apenas a segunda posição do vetor.

Pai (antes da mutação): [5.5, 2.0, 9.1]

Posição sorteada para mutar: Índice 2.

Novo valor sorteado (entre 0 e 20): 17.4

Filho Mutante resultante: [5.5, 17.4, 9.1]

c. **Mutação Adaptativa Viável** (*mutationadaptfeasible*)

Esta é a função do MATLAB usada quando o problema de otimização possui restrições (limites estritos ou equações lineares).

Aplicar um ruído Gaussiano ou Uniforme poderia jogar o indivíduo mutante para fora da área permitida do problema (uma zona inviável). Para evitar isso, o método Adaptativo Viável gera direções de mutação aleatórias, mas verifica os limites. Se o passo de mutação for bater em uma restrição, o algoritmo reduz o tamanho desse passo de forma adaptativa para garantir que o filho nasça dentro da região viável. Além disso, se a mutação em uma certa direção gerar filhos com notas melhores (*fitness*), o algoritmo aumenta a probabilidade de continuar mutando nessa direção nas próximas gerações.

Exemplo:

Considere que a restrição do problema dita que as variáveis não podem passar de 10.0.

Pai (perto da restrição): [9.5, 5.0]

Direção do ruído gerado: [+2.0, 0.0] (levaria a coordenada 1 para 11.5, violando a restrição).

Ação Adaptativa: detecta o limite, encolhe o passo para não ultrapassar 10.0.

Filho Mutante resultante: Pode nascer em $[9.9, 5.0]$, encostando no limite, mas sem quebrá-lo.

5. Conclusão

A implementação do Algoritmo Genético pelo solver do MATLAB representa uma adaptação da teoria de algoritmos genéticos clássica para o cenário de otimização na engenharia. Na literatura padrão, os GA são frequentemente modelados através de cromossomos binários (cadeias de bits e *nibbles*), onde o *crossover* consiste no corte e união direta de fitas de código e a mutação, uma simples inversão de um bit (de 0 para 1). Além disso, a teoria clássica costuma aplicar a mutação sequencialmente sobre o filho recém-gerado pelo cruzamento.

O MATLAB, visando a eficiência em problemas contínuos e complexos, além de adotar uma estrutura vetorial, faz uma adaptação dessa teoria para a sua necessidade. A principal adaptação da ferramenta foi estruturar a população inteiramente em vetores de números reais. Com essa mudança, a estrutura cromossômica biológica é substituída por uma estrutura algébrica. Os cortes genéticos são substituídos por operações lógicas, médias ponderadas e extrapolações vetoriais (*crossover* disperso, aritmético, heurístico). A mutação se torna uma injeção de ruído estatístico (Gaussiano ou uniforme). Por fim, ele inova ao separar a reprodução na estratégia restrita de *crossover* ou mutação, garantindo caminhos matemáticos independentes para a convergência e a manutenção da diversidade. Toda essa adaptação consolida o solver como uma ferramenta capaz de evitar mínimos locais de forma ágil e encontrar soluções globais em superfícies não-lineares reais.

6. Referências

MATHWORKS. Global Optimization Toolbox: Genetic Algorithm. Natick, MA: The MathWorks, Inc. Documentação oficial. Disponível em: <https://www.mathworks.com/help/gads/genetic-algorithm.html>. Acesso em: 28 jul. 2026.

MATHWORKS. Global Optimization Toolbox: Crossover Options. Natick, MA: The MathWorks, Inc. Documentação oficial. Disponível em: <https://www.mathworks.com/help/gads/crossover-options.html>. Acesso em: 28 jul. 2026.

MATHWORKS. Global Optimization Toolbox: Mutation Options. Natick, MA: The MathWorks, Inc. Documentação oficial. Disponível em: <https://www.mathworks.com/help/gads/mutation-options.html>. Acesso em: 28 jul. 2026.